

Taller: Benchmarking de APIs con Postgres usando JMeter

Trabajo en grupo de **4 estudiantes**, pero **cada integrante trabaja en su propio computador**, con su **propio proyecto** y su **propio docker-compose**.

1. Elige tu stack

Dentro del grupo, cada estudiante debe escoger **uno diferente**:

- Python + Flask
- Node.js + NestJS
- Java + Spring Boot
- Go o Rust

2. Crea tu microservicio

Implementa en tu stack el endpoint: **POST /api/orders**

Entrada (JSON):

```
{
  "customerId": "C123",
  "items": [
    { "productId": "P1", "quantity": 2, "price": 10.5 },
    { "productId": "P2", "quantity": 1, "price": 5.0 }
  ]
}
```

Tu API debe:

1. Calcular totalAmount y itemCount.
2. Insertar los datos en **PostgreSQL** en dos tablas:
 - a. orders
 - b. order_items
3. Retornar un JSON con el orderId, totales y fecha de procesamiento.

3. Configura tu base de datos

En tu Postgres (en Docker) crea estas tablas:

```
CREATE TABLE orders (  
  id SERIAL PRIMARY KEY,  
  order_id VARCHAR(50) UNIQUE NOT NULL,  
  customer_id VARCHAR(50) NOT NULL,  
  total_amount NUMERIC(10,2) NOT NULL,  
  items_count INT NOT NULL,  
  created_at TIMESTAMP NOT NULL DEFAULT NOW()  
);
```

```
CREATE TABLE order_items (  
  id SERIAL PRIMARY KEY,  
  order_id VARCHAR(50) NOT NULL,  
  product_id VARCHAR(50) NOT NULL,  
  quantity INT NOT NULL,  
  price NUMERIC(10,2) NOT NULL  
);
```

4. Crea tu docker-compose

Cada integrante debe tener un **docker-compose.yml** con:

- Un contenedor para tu API
- Un contenedor para PostgreSQL

Ejemplo:

```
version: "3.9"  
services:  
  api:  
    build: .  
    container_name: api-orders  
    depends_on:  
      - db  
    environment:  
      - DB_HOST=db  
      - DB_PORT=5432  
      - DB_USER=postgres  
      - DB_PASSWORD=postgres  
      - DB_NAME=orders_db  
    ports:  
      - "5001:5001"
```

db:

image: postgres:16

container_name: db-orders

environment:

- POSTGRES_USER=postgres
- POSTGRES_PASSWORD=postgres
- POSTGRES_DB=orders_db

ports:

- "5433:5432"

volumes:

- dbdata:/var/lib/postgresql/data

volumes:

dbdata:

Levanta tu entorno con:

`docker compose up -d`

Comprueba que tu API responde en: POST http://localhost:<tu_puerto>/api/orders

5. Crea tu Test Plan en JMeter

(a) Abre JMeter → **Test Plan**

(b) Añade un **Thread Group**

(c) Configura 3 escenarios de carga:

- Baja: 20 usuarios
- Media: 70 usuarios
- Alta: 150 usuarios

(d) Añade:

- HTTP Request Defaults** (server: localhost, path: /api/orders)
- HTTP Header Manager** (Content-Type: application/json)
- HTTP Request** → método **POST**, con el JSON del pedido

(e) Añade listeners:

- Summary Report**
- Aggregate Report**

Ejecuta las pruebas **contra tu API**, en tu máquina.

6. Registra tus resultados

Por cada escenario guarda:

1. Average response time
2. Percentil 95
3. Throughput
4. % de errores

Resultados individuales

Cada estudiante entrega un archivo o tabla con:

a) Métricas por escenario:

Para **cada nivel de carga**:

- **Baja carga:** 10 usuarios
- **Media carga:** 50 usuarios
- **Alta carga:** 100 usuarios
- **Escenario aumentando de 20 en 20 hasta notar estrés**

Debe registrar:

1. **Average Response Time (ms)**
2. **P95 o P90** (percentil 95 o 90)
3. **Throughput (requests/second)**
4. **% Error**
5. **Observaciones personales** (si hubo cuellos de botella, timeouts, fallas, etc.)

b) Evidencias:

- Captura de pantalla del **Summary Report**
- Captura del **Aggregate Report**
- Pantallazo de JMeter durante la carga
- Evidencia del contenedor corriendo (docker ps)
- Monitoreo de procesos durante cada prueba utilizando **htop** y **docker stats**

Entrega Grupal

El grupo debe crear una **tabla comparativa única** con los resultados de los 4 stacks.

- 1. Esa tabla debe tener:
- 2. 4 filas (1 por stack)
- 3. 3 columnas de escenarios (baja, media, alta)
- 4. Subcolumnas para cada métrica

Ejemplo de tabla consolidada

Consumo del sistema (por stack y escenario)

Escenario	CPU API (%)	RAM API (MiB)	CPU DB (%)	RAM DB (MiB)	Observaciones
Baja carga (10)	12%	90 MiB	5%	70 MiB	Estable
Media carga (50)	55%	180 MiB	18%	100 MiB	Sin errores
Alta carga (100)	92%	350 MiB	30%	150 MiB	Latencias altas después del segundo minuto

Comparación del desempeño por stack y escenario

Baja carga (10 usuarios)

Stack	Avg (ms)	P95 (ms)	Throughput (req/s)	% Error
Flask (Python)	22	35	450	0%
NestJS (Node)	18	27	520	0%
Spring Boot (Java)	9	13	880	0%
Go	5	8	1100	0%

Media carga (50 usuarios)

Stack	Avg (ms)	P95 (ms)	Throughput (req/s)	% Error
Flask (Python)	105	180	310	2%
NestJS (Node)	72	120	430	1%
Spring Boot (Java)	25	40	800	0%
Go o Rust	12	18	1000	0%

Alta carga (100 usuarios)

Stack	Avg (ms)	P95 (ms)	Throughput (req/s)	% Error
Flask (Python)	260	380	210	5%
NestJS (Node)	180	290	320	3%
Spring Boot (Java)	55	80	700	1%
Go o Rust	25	40	900	0%

Estrés (? usuarios)

Stack	Avg (ms)	P95 (ms)	Throughput (req/s)	% Error
Flask (Python)	260	380	210	5%
NestJS (Node)	180	290	320	3%
Spring Boot (Java)	55	80	700	1%
Go o Rust	25	40	900	0%

Conclusiones que debe escribir el grupo

El informe debe responder estas preguntas:

Rendimiento

1. ¿Cuál stack tuvo **mejor latencia promedio**?
2. ¿Cuál mantuvo el **mejor throughput** en cargas altas?
3. ¿Qué lenguaje fue más estable?

Eficiencia

4. ¿Algún stack mostró **errores** bajo alta carga?
5. ¿Qué puede explicar esas fallas? (GC, ORM lento, thread model, etc.)

Recomendación para producción

6. ¿Cuál stack recomendarían para una empresa que requiere alto desempeño?
7. ¿Cuál stack es más fácil de implementar pero sacrifica rendimiento?
8. ¿Cuál stack ofrece un mejor balance entre sencillez y velocidad?